

First-Step Baselines

Temporal-shift ML benchmark for CTR prediction, before developing a new method

Goal: build a reliable chronological CTR benchmark for established temporal-adaptation methods. Do not implement a new method or a new autobidding mechanism in this step.

Project context. This step narrows the earlier autobidding proposal: the ML problem is temporal distribution shift; CTR is the primary prediction task; autobidding will be added only after the prediction benchmark is stable, using the same standard bidder for every predictor.

1. Scope and success criterion

- Primary task: one-step-ahead CTR prediction under chronological distribution shift.
- Primary comparison: how different rules for using historical data affect future-period prediction.
- Base model first: regularized logistic regression. Keep architecture fixed while comparing temporal methods.
- Autobidding is out of scope until the prediction benchmark, splits, metrics, and baselines are frozen.
- The step succeeds when all required baselines run from a common interface, produce daily metrics, and pass leakage/reproducibility checks.

2. Required baseline ladder

Method	Historical-data rule	Role	Priority
Expanding ERM	All available history	Stationary / maximum-data baseline	P0
Rolling windows	Last 1, 3, 7, 14 days	Classic recency baselines	P0
Exponential forgetting	Age-decayed weights; 3 fixed half-lives	Smooth recency baseline	P0
Validation-selected window	Choose among fixed windows on development data, then freeze	Strong practical baseline	P0
Han et al. adaptive rolling window	Data-dependent global history length	General temporal-shift baseline	P1
Differentiable Forgetting	Learn parameters of temporal weighting	Learned forgetting baseline	P1
AdaMoE or SFTL	CTR-specific drift adaptation	Specialized CTR reference	P2

Implementation order: finish every P0 method before P1; finish P1 before adding a CTR-specific P2 model. Do not add extra baselines merely because code is available.

3. Baseline definitions

3.1 Expanding-history ERM

At prediction period t , train on every permissible earlier period:

$$D_{\text{train}}(t) = D_1 \cup D_2 \cup \dots \cup D_{t-1}.$$

Fit the same regularized logistic-regression objective used by all P0 methods. This baseline answers whether maximum sample size is better than recency when the environment drifts.

3.2 Fixed rolling windows

Run four separate methods with $h \in \{1, 3, 7, 14\}$ days:

$$D_{\text{train}}^{(h)}(t) = D_{t-h} \cup \dots \cup D_{t-1}.$$

If fewer than h historical days are available, use all available prior days. These are separate baselines; do not choose h using test performance.

3.3 Exponential forgetting

Use all prior examples but weight period s by its age:

$$w_{\{t,s\}} = \exp[-\gamma (t-s)].$$

Parameterize γ by half-life. Use a compact predeclared grid such as 1, 3, and 7 days. Tune only on the development period; never on locked test days.

3.4 Validation-selected window

On development folds only, select the best member of {1-day, 3-day, 7-day, 14-day, expanding}. Freeze that choice before the final test. This is the practical baseline to beat, not an oracle that re-selects h on each test day.

3.5 Han et al. adaptive rolling window

Reproduce the authors' adaptive rolling-window method before adapting it to CTR. Preserve its statistical selection rule rather than replacing it with a heuristic. Integrate it through the same train/predict interface and record the selected effective window for every prediction day. Reference: [Han, Huang & Wang, ICML 2024](#)

3.6 Differentiable Forgetting

Implement a learned temporal weighting rule following the published formulation. The outer objective must be evaluated on later chronological data, not a random validation split. Keep the base predictor comparable to the simpler baselines when feasible. Reference: [Differentiable Forgetting](#)

3.7 CTR-specific adaptive baseline

Add only after P0/P1 are stable. Prefer AdaMoE first; SFTL is an alternative if code/data compatibility is better. Treat this as a specialized reference, not as the common base-model comparison. References: [AdaMoE](#) | [SFTL, AAAI 2024](#)

4. Data and chronological protocol

Primary dataset: Criteo Attribution Modeling for Bidding. Use the click label as the CTR target. Preserve original time order and day/campaign identifiers. Outcome-side variables that would not be available at bidding time must not be model inputs.

- Partition time into development and locked test periods. Exact boundaries should be specified once the raw timestamps are audited.
- Within development, use rolling-origin validation: train strictly on periods before the validation day.
- Locked test evaluation must simulate deployment: at day t , only data that would have been observed before t may be used.

- No random train/test split, no random cross-validation across time, and no preprocessing fitted on future test data.
- If categorical encoders or scalers need fitted statistics, fit them using permissible history only, or use a declared vocabulary policy that cannot leak future label information.

Leakage rule: for a prediction on period t , every learned quantity - model parameters, hyperparameters, sample weights, encoders, and window choices - must depend only on information available before t .

5. Metrics and required diagnostics

- Primary: log loss. This should drive the principal prediction comparison.
- Secondary: Brier score and PR-AUC; ROC-AUC may be reported but should not dominate the story.
- Report every metric both aggregated and by prediction day.
- Record training-set size / effective sample size, training time, and inference time for each method.
- For methods that select or learn memory, record the selected window, half-life, or effective temporal weight profile over time.

Critical diagnostic: hindsight best fixed window. For each test day, retrospectively identify which fixed candidate window achieved the best loss on that day. This is for analysis only and must never feed the deployed method.

$$h^*_t = \operatorname{argmin}_h L_t(h), \quad h \in \{1, 3, 7, 14, \text{expanding}\}.$$

Plot h^*_t over time. If the best window changes materially, there is empirical motivation for adaptive memory. If one fixed horizon dominates almost everywhere, the future research direction should be reconsidered.

6. Implementation architecture

Use one common interface so temporal data selection is separated from the base predictor. A suggested structure:

```
data/      load, validate, timestamp, split, feature schema
models/    logistic regression + later CTR architectures
methods/   expanding, rolling, decay, val_best, han_arw, diff_forgetting
runner/    rolling-origin train -> predict -> log metrics
metrics/   logloss, Brier, PR-AUC, per-day aggregation
analysis/  best-window diagnostic, drift plots, significance summaries
configs/   immutable YAML/TOML experiment specifications
tests/     leakage, split, deterministic seed, unit tests
```

Every temporal method should expose conceptually:

$$\begin{aligned} \text{fit}(\text{history_before_t}, \text{config}) &\rightarrow \text{model_t} \\ \text{predict}(\text{model_t}, X_t) &\rightarrow \hat{p}_t \\ \text{metadata}(\text{model_t}) &\rightarrow \text{selected history / weights / diagnostics} \end{aligned}$$

7. Execution sequence

1. Audit the Criteo files: timestamps, day boundaries, click prevalence, campaign counts, missing values, and features available before bidding.
2. Freeze chronological development/test splits and write automated assertions that no training row is later than its prediction period.
3. Build the shared logistic-regression pipeline and reproduce expanding-history ERM.

4. Add rolling 1/3/7/14-day baselines and verify that changing h only changes permissible training rows.
5. Add exponential forgetting with a small development-only half-life grid.
6. Add validation-selected window and ensure the selected h is frozen before locked test evaluation.
7. Generate the first comparison table and per-day loss plots. Compute the hindsight best-window diagnostic.
8. Reproduce Han et al. on a small controlled example or their released setup, then integrate it into the CTR runner.
9. Implement Differentiable Forgetting and verify its outer validation chronology.
10. Only if the benchmark is stable, add AdaMoE or SFTL as the CTR-specific reference.

8. Acceptance tests

Check	Pass condition
Chronology	No training or tuning observation is from the prediction period or future.
Feature leakage	No outcome-side or post-bid feature enters the CTR model.
Shared model	P0 methods use the same logistic-regression implementation and feature pipeline.
Hyperparameters	All tuning uses development periods only; locked test is evaluated once after freeze.
Reproducibility	Same config + seed reproduces predictions and aggregate metrics within numerical tolerance.
Daily logging	Every run stores per-day prediction metrics and temporal-method metadata.
Baseline sanity	Expanding/rolling/decay results are plausible on a small hand-checked slice before full runs.

9. Deliverables from the first step

- A reproducible chronological CTR benchmark runner.
- One configuration file per method and a single command to reproduce each baseline.
- Table: aggregate log loss, Brier score, PR-AUC, training time, and effective history size.
- Figure: per-day log loss for all P0 methods.
- Figure: hindsight best fixed window h^*_t over time.
- Figure/table: selected or learned memory behavior for Han ARW and Differentiable Forgetting.
- Short findings note documenting when recency helps, when long history helps, and where adaptive methods fail.

10. Go / no-go gate before inventing a new method

Proceed to new-method design only after the benchmark reveals a concrete, repeatable weakness of existing temporal adaptation methods.

- Strong motivation: the hindsight best horizon changes across time or subperiods, and existing adaptive methods fail to track that variation.

- Strong motivation: a method improves average prediction but recovers slowly after abrupt or recurring shifts.
- Strong motivation: prediction metrics and downstream bidding value later disagree in a systematic way.
- No-go signal: one simple fixed window or fixed decay rate dominates across essentially all periods and datasets.

11. What comes after this step

After prediction baselines are frozen, add a deliberately standard autobidding evaluation. Feed every CTR predictor into the same bidding/pacing rule and compare realized value at matched spend. Do not change the bidder across ML methods. The autobidding stage should test the consequence of temporal-shift robustness, not introduce a second algorithmic contribution.

CTR method $\rightarrow \hat{p}_t \rightarrow$ same standard bidder $\rightarrow$ value at matched spend.

References for implementation

[Han, Huang, and Wang. Model Assessment and Selection under Temporal Distribution Shift. ICML 2024.](#)

[Bennett and Clarkson. Differentiable Forgetting.](#)

[Liu et al. On the Adaptation to Concept Drift for CTR Prediction \(AdaMoE\).](#)

[Zhu et al. Generalize for Future: Slow and Fast Trajectory Learning for CTR Prediction. AAAI 2024.](#)

[Criteo Attribution Modeling for Bidding Dataset.](#)